Advanced Audio Processing: Exercise 04

Audio Classification with Data Augmentation

Emmanuel Safo
Student ID: 153058679

February 2026

1 Introduction

This report presents the results of implementing and evaluating data augmentation techniques for audio classification using the ESC-50 dataset subset. The dataset contains four classes: *rain*, *sea_waves*, *chainsaw*, and *helicopter*. A Convolutional Neural Network (CNN) was trained with various augmentation strategies to investigate their impact on classification performance.

2 Implemented Augmentation Techniques

Four data augmentation methods were implemented:

1. **Noise Addition**: White Gaussian noise added at a specified Signal-to-Noise Ratio (SNR) of 15 dB. The noise power is calculated as:

$$P_{noise} = \frac{P_{signal}}{10^{SNR_{dB}/10}}$$

2. **Pitch Shifting**: Audio pitch shifted by a random number of semitones in the range $[-2, +2]$ using librosa's pitch shift function.

3. **Reverberation**: Convolution with Room Impulse Responses (RIRs) to simulate different acoustic environments. Five different impulse responses were used.

4. **Frequency Masking (SpecAugment)**: Random consecutive frequency bands (up to 20 bands) in the mel spectrogram are zeroed out, following the SpecAugment technique.

2.1 Code Implementation: Data Augmentation Functions

The following code shows the implementation of the four augmentation functions in `data_augmentation_templa`

Listing 1: Noise Addition Function

```python
def add_noise(y: np.ndarray, snr_db: float = 20.0) -> np.ndarray:
    # Calculate signal power
    signal_power = np.mean(y ** 2)

    # Calculate noise power needed for desired SNR
    noise_power = signal_power / (10 ** (snr_db / 10))

    # Generate white noise with the calculated power
    noise = np.random.normal(0, np.sqrt(noise_power), len(y))

    # Add noise to signal
    y_noised = y + noise
    return y_noised
```

Listing 2: Pitch Shifting Function

```python
def add_pitch_shift(y: np.ndarray, n_steps: float, sr: int = 44100) -> np.ndarray:
    y_shift = librosa.effects.pitch_shift(y=y, sr=sr, n_steps=n_steps)
    return y_shift
```

Listing 3: Room Impulse Response (Reverberation) Function

```python
def add_impulse_response(y: np.ndarray, impulse_response: np.ndarray) -> np.ndarray:
    # Convolve signal with impulse response using FFT-based convolution
    y_impulse = fftconvolve(y, impulse_response, mode='full')

    # Normalize to prevent clipping
    max_val = np.max(np.abs(y_impulse))
    if max_val > 0:
        y_impulse = y_impulse * (np.max(np.abs(y)) / max_val)
    return y_impulse
```

Listing 4: Frequency Masking (SpecAugment) Function

```python
def freq_masking(y: np.ndarray, max_bands: Optional[int] = None) -> np.ndarray:
    y_masking = y.copy()
    n_mels = y.shape[0]  # Number of mel frequency channels

    if max_bands is None:
        max_bands = min(40, n_mels)
    max_bands = min(max_bands, n_mels)

    # Sample f uniformly from [0, max_bands]
    f = np.random.randint(0, max_bands + 1)

    # Sample f0 uniformly from [0, n_mels - f]
    if n_mels - f > 0:
        f0 = np.random.randint(0, n_mels - f)
    else:
        f0 = 0

    # Apply mask: zero out frequency channels [f0, f0+f)
    y_masking[f0:f0 + f, :] = 0
    return y_masking
```

3 Experimental Setup

Table 1: Training Configuration

Parameter	**Value**
Batch Size	4
Learning Rate	1×10^{-4}
Optimizer	Adam
Loss Function	Cross-Entropy
Max Epochs	30
Early Stopping Patience	10
Mel Bands	64
Sample Rate	44100 Hz

3.1 Code Implementation: Dataset Class

The __getitem__ method in `dataset_class_template.py` loads audio, applies augmentations, and extracts features:

Listing 5: Dataset __getitem__ Method

```python
def __getitem__(self, item: int) -> Tuple[np.ndarray, np.ndarray]:
    # Read the audio file
    audio_path = self.audio_file_paths[item]
    y, sr = utils.get_audio_file_data(str(audio_path))
    y_len = len(y)
    y_type = y.dtype

    # Apply time domain augmentations
    if self.pitch_shift_augmentation:
        n_steps = random.uniform(-self.pitch_shift_max_steps,
                                 self.pitch_shift_max_steps)
        y = data_augmentation.add_pitch_shift(y, n_steps, sr)
    if self.reverb_augmentation:
        impulse_response = random.choice(self.impulses)
        y = data_augmentation.add_impulse_response(y, impulse_response)
    if self.noise_augmentation:
        y = data_augmentation.add_noise(y, self.snr_db)

    # Ensure original length and type
    y = y[:y_len].astype(y_type)

    # Extract mel band energies
    mels = utils.extract_mel_band_energies(y, sr=self.sr, n_fft=self.n_fft,
                                           hop_length=self.hop_length,
                                           n_mels=self.n_mels)

    # Apply frequency domain augmentations
    if self.freqmask_augmentation:
        mels = data_augmentation.freq_masking(mels, self.max_bands)

    # Get label and encode with one-hot
    filename = audio_path.name
    label = self.meta_df[self.meta_df['filename'] == filename]['label'].values
        [0]
    cls_vector = utils.create_one_hot_encoding(label, self.labels)

    return mels, cls_vector
```

3.2 Code Implementation: Training Loop

The training script in `training_template.py` implements the training, validation, and testing loops:

Listing 6: Training Loop (Excerpt)

```python
for epoch in range(epochs):
    model.train()
    for i, batch in enumerate(train_loader):
        features, targets = batch
        features = features.float().to(device)
        targets = targets.float().to(device)

        optimizer.zero_grad()
        outputs = model(features)
        loss = loss_function(outputs, targets)
        loss.backward()
```

```
        optimizer.step()

        # Calculate accuracy
        predicted = torch.argmax(outputs, dim=1)
        true_labels = torch.argmax(targets, dim=1)
        epoch_acc_training += (predicted == true_labels).sum().item()

    # Validation loop
    model.eval()
    with torch.no_grad():
        for i, batch in enumerate(val_loader):
            features, targets = batch
            features = features.float().to(device)
            targets = targets.float().to(device)
            outputs = model(features)
            loss = loss_function(outputs, targets)

    # Early stopping check
    if epoch_loss_validation < lowest_validation_loss:
        lowest_validation_loss = epoch_loss_validation
        best_model = deepcopy(model.state_dict())
        patience_counter = 0
    else:
        patience_counter += 1
```

4 Results

4.1 Model Performance

The model was trained with all augmentation techniques enabled on the training set, while validation and test sets used only the original (non-augmented) data to ensure fair evaluation.

Table 2: Training Results with Combined Augmentations

Metric	**Value**
Best Validation Loss	1.2354
Best Epoch	14
Test Loss	1.1376
Test Accuracy	**58.3%**

4.2 Confusion Matrix Analysis

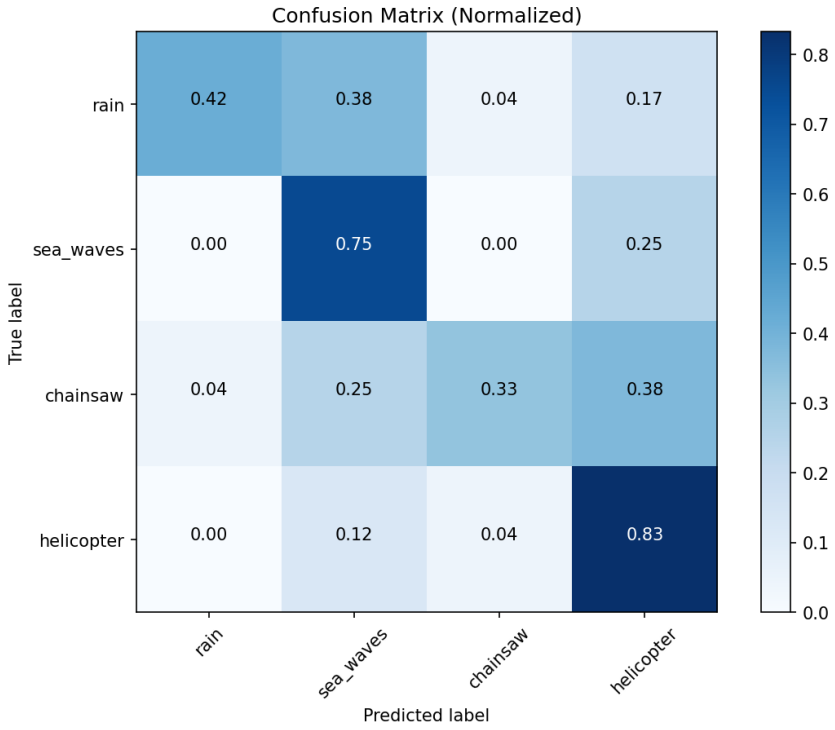

Figure 1: Normalized Confusion Matrix for Test Set

The confusion matrix reveals class-specific performance:

- **Helicopter**: Best classified with 83.3% accuracy
- **Sea Waves**: Good performance at 75.0% accuracy
- **Rain**: Moderate performance at 41.7% accuracy, often confused with sea waves
- **Chainsaw**: Lower accuracy at 33.3%, confused with helicopter and sea waves

The confusion between *rain* and *sea_waves* is expected as both are ambient environmental sounds with similar spectral characteristics.

5 Discussion

5.1 Impact of Data Augmentation

The combination of all four augmentation techniques helped the model achieve reasonable performance (58.3%) despite the limited training data (only 8 samples per class). The augmentations work as follows:

- **Noise addition** helps the model become robust to background noise variations in real-world recordings.
- **Pitch shifting** exposes the model to frequency variations, useful for sounds that may vary in pitch.
- **Reverberation** simulates different recording environments, improving generalization across acoustic conditions.

- **Frequency masking** acts as regularization, preventing the model from over-relying on specific frequency bands.

5.2 Combination vs. Single Method

A combination of augmentation techniques was used rather than a single method. This approach is beneficial because:

1. Each augmentation addresses different sources of variation in audio data
2. The combination creates more diverse training samples, reducing overfitting
3. Different augmentations complement each other—time-domain (noise, pitch, reverb) and frequency-domain (masking) augmentations target different aspects of the signal

6 Conclusion

Data augmentation proved effective for audio classification with limited training data. The combination of noise addition, pitch shifting, reverberation, and frequency masking achieved 58.3% test accuracy on a 4-class classification problem. The *helicopter* class was best recognized, while ambient sounds (*rain*, *sea_waves*) showed some confusion due to spectral similarity. Future work could explore additional augmentation techniques such as time stretching or more aggressive SpecAugment parameters.